SIH PROTOTYPE EXECUTION GUIDE

Viraasat AI Prototype Flow

Next.js frontend + Python FastAPI backend + Supabase

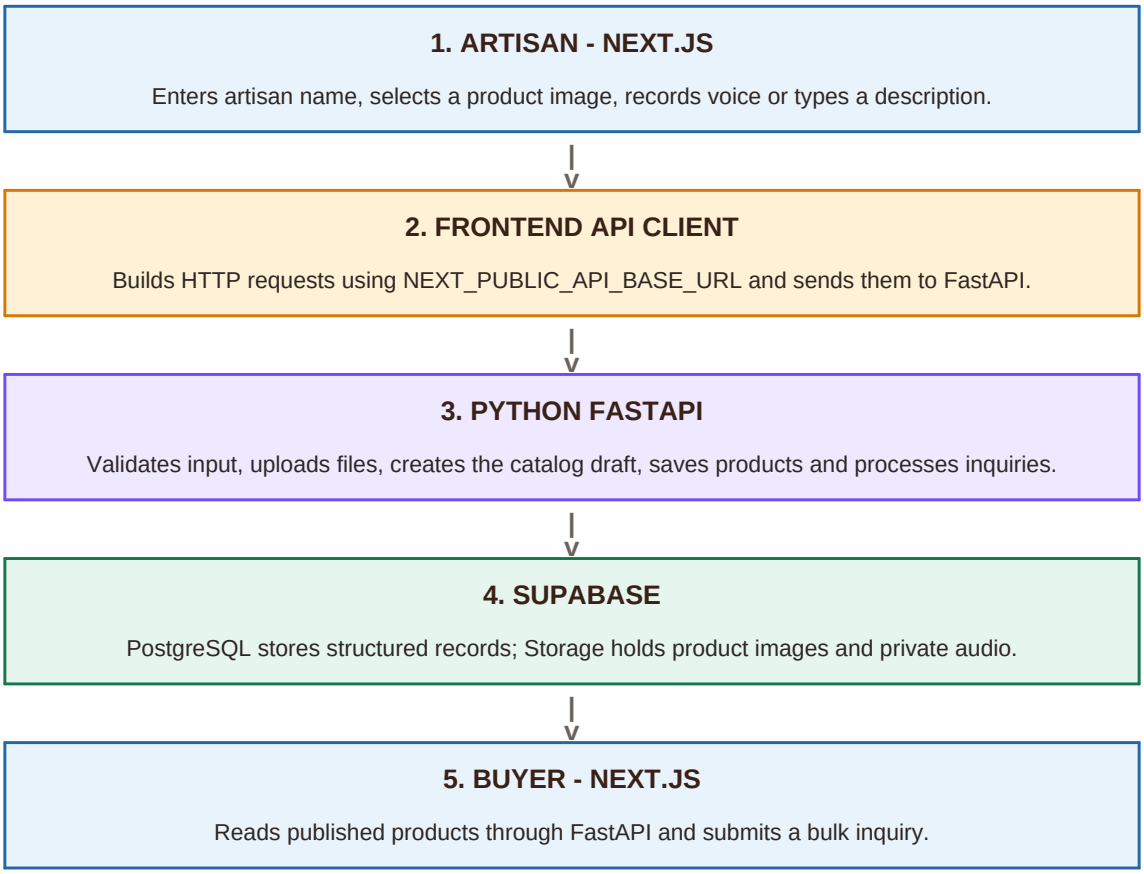
Version 1 goal: Upload a product photo and voice/text description, generate an editable listing, publish it, display it in the marketplace, and accept a buyer inquiry.

Frontend	Backend	Data
Next.js handles screens, forms, recording, loading states and result display.	FastAPI owns every API, validation rule, upload and business operation.	Supabase stores products, inquiries, images and private audio.

Prepared for the four-member Viraasat AI technical team

1. Prototype at a glance

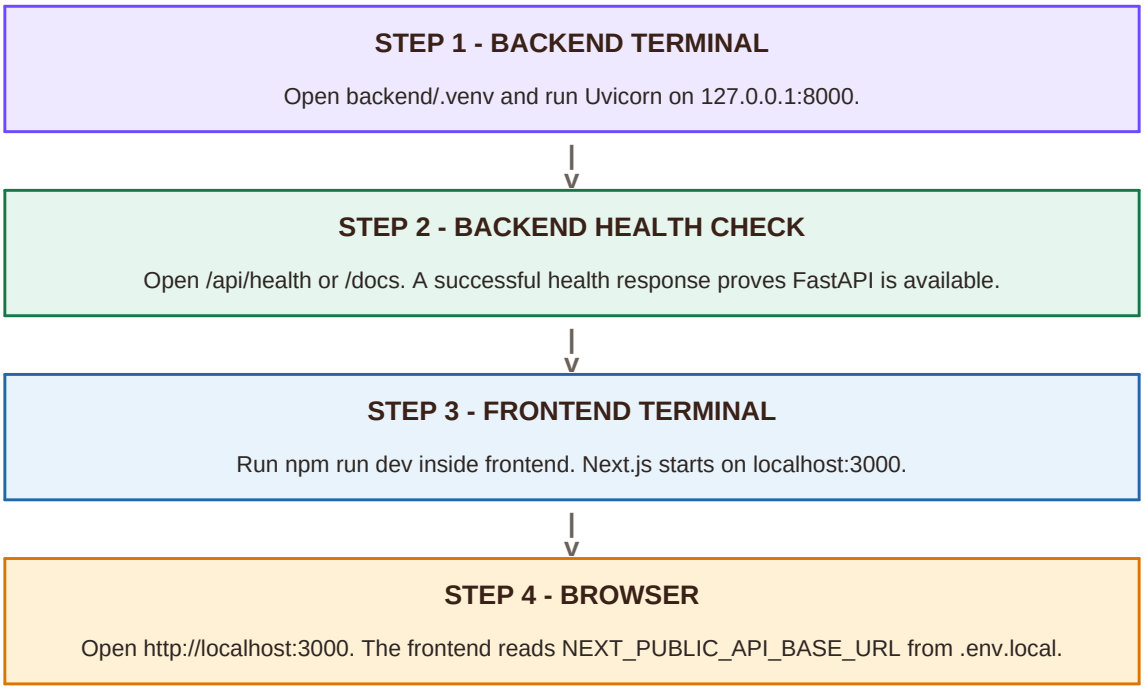
The prototype is a single end-to-end path. Every layer has one clear responsibility, and the browser never accesses Supabase directly.



Core boundary: Next.js -> FastAPI -> Supabase. No Supabase service-role key is ever placed in the frontend.

2. Local startup flow

Both services must run at the same time. Start FastAPI first, then Next.js.



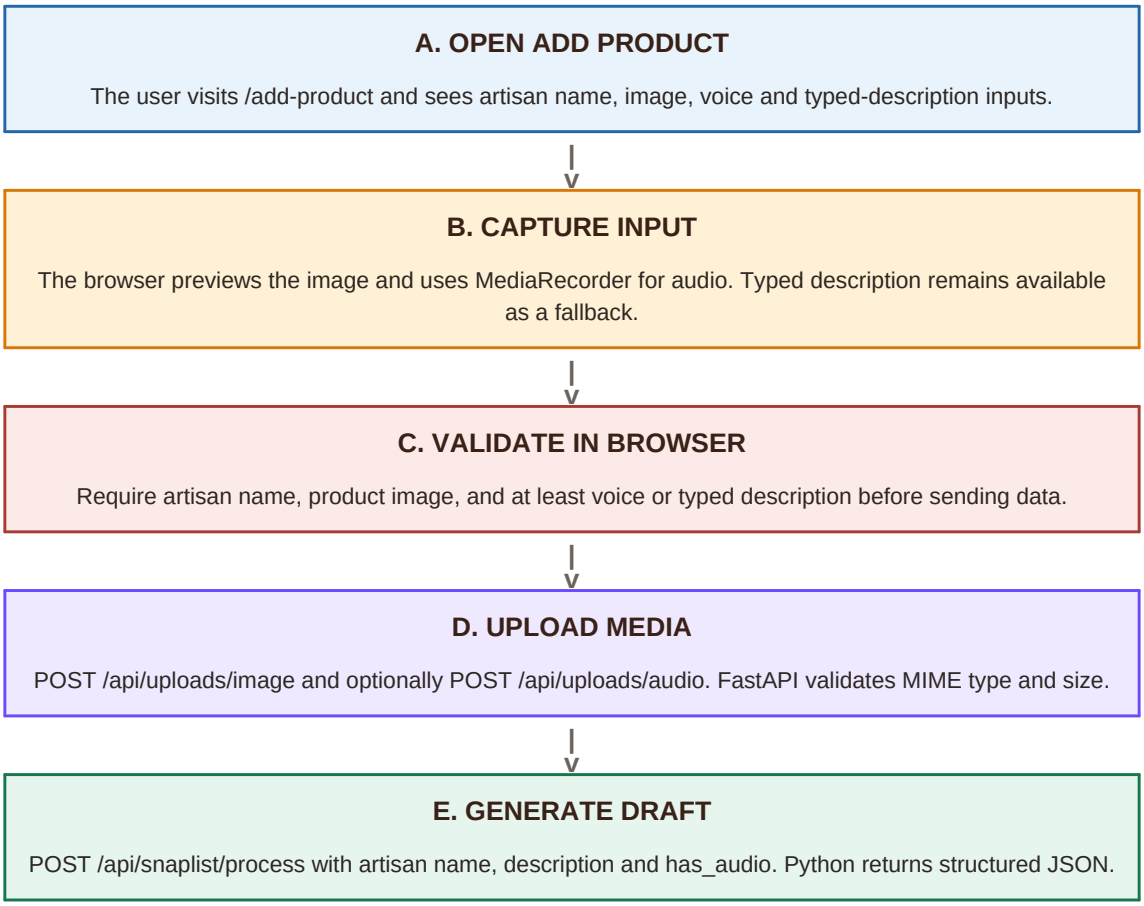
Environment files

File	Required values	Visibility
frontend/.env.local	NEXT_PUBLIC_API_BASE_URL=http://localhost:8000	Visible to browser
backend/.env	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, FRONTEND_ORIGINS	Backend only

If backend/.env is missing, the health check and mock catalog route work, but uploads, products and inquiries return a configuration error.

3. Artisan capture and generation flow

This is the first half of the live demonstration, implemented on the Add Product page.



Version 1 behavior

- The generator is deterministic Python logic, not a production AI model.
- Known phrases such as blue pottery, brass or textile return useful sample listings.
- A generic listing is returned when no known category is detected.
- Real STT and VLM services can later replace the mock service without changing the frontend route.

4. Request and response sequence

The frontend API client joins the backend base URL with each route. FastAPI validates every request with Pydantic.

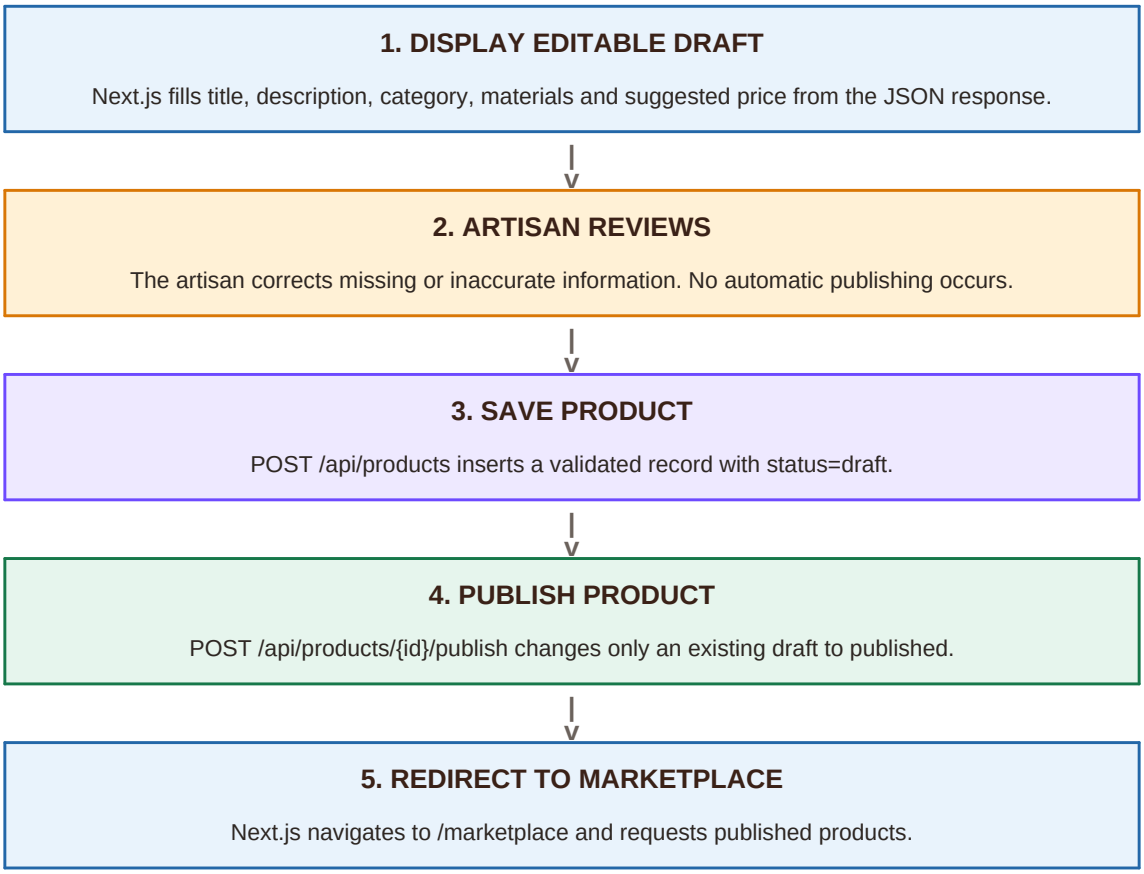
Order	Frontend action	FastAPI route	Result
1	Upload image	POST /api/uploads/image	Public product image URL
2	Upload audio	POST /api/uploads/audio	Private storage path
3	Generate listing	POST /api/snaplist/process	CatalogDraft JSON
4	Save reviewed draft	POST /api/products	Product with UUID and draft status
5	Publish	POST /api/products/{id}/publish	Published product
6	Open marketplace	GET /api/products	Published products only
7	Open product	GET /api/products/{id}	Single published product
8	Send inquiry	POST /api/inquiries	Saved buyer inquiry

Next.js	FastAPI	Supabase
Calls apiRequest() and updates React state from returned JSON.	Routes, validates and delegates to Python services.	Returns records or storage locations to FastAPI.

The backend responds first; only then does the frontend move to the next screen or update the current screen.

5. Review, save and publish flow

AI or mock output is always treated as a draft. Human review is a required control before the product becomes public.

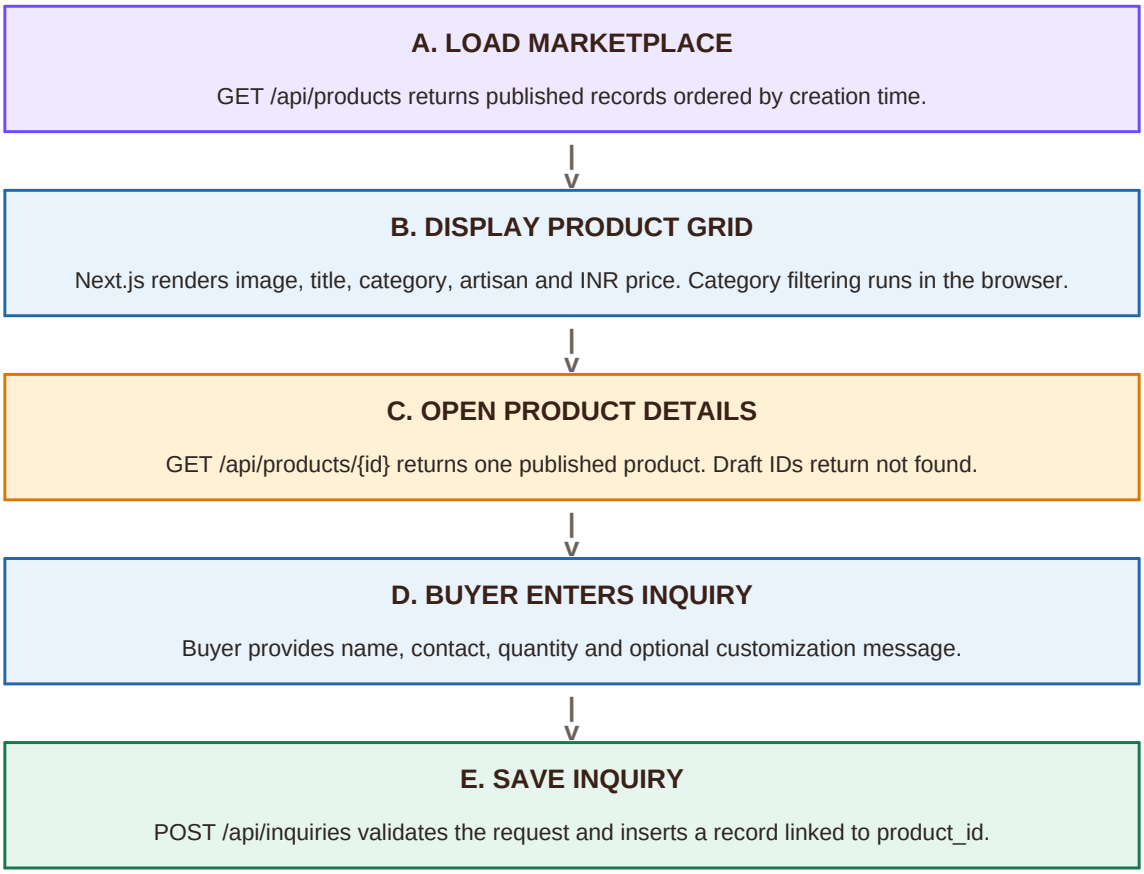


Why two backend calls?

- Creating the draft gives the product a permanent UUID.
- Publishing is a separate business action and can later require authentication or admin approval.
- The marketplace query filters by status=published, so unfinished drafts remain hidden.

6. Marketplace and buyer inquiry flow

The second half of the prototype demonstrates market linkage after an artisan publishes a product.



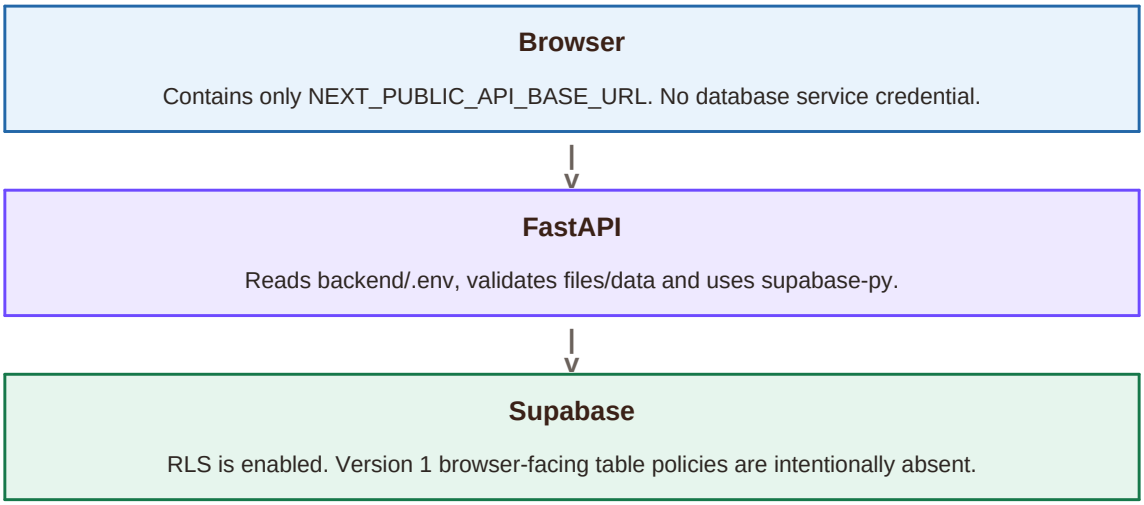
Version 1 stops at a saved inquiry. Payments, production batches and fulfilment are intentionally outside the basic prototype.

7. Supabase data and storage flow

FastAPI is the only application layer that possesses the Supabase service-role key.

products	inquiries	Storage
Structured listing, artisan, category, materials, price, image/audio references and status.	Buyer name/contact, quantity, message and product UUID relationship.	Public product-images bucket and private product-audio bucket.

Security boundary



- Product images are public because marketplace cards need to display them.
- Raw audio remains private and is stored as a backend-controlled path.
- Image limit: 8 MB. Audio limit: 12 MB. MIME type is checked server-side.
- Do not commit backend/.env or frontend/.env.local.

8. Failure handling and demo script

A clear failure path prevents the prototype from appearing broken when input, configuration or connectivity is incomplete.

Failure	Backend/frontend behavior	User action
Missing required input	Frontend blocks submit; Pydantic also returns 422.	Add the missing field.
Unsupported/large media	Upload API returns a safe validation error.	Choose a supported smaller file.
Backend not running	Frontend fetch fails and shows an error state.	Start Uvicorn on port 8000.
Supabase not configured	Database/upload routes return BACKEND_NOT_CONFIGURED.	Create backend/.env and run migration.
No products	Marketplace displays an empty state.	Publish the first product.
Unknown product	Product route returns 404.	Return to marketplace.

Recommended 3-minute prototype demonstration

Time	Presenter action
0:00-0:25	Explain the artisan problem and the five-screen prototype.
0:25-1:05	Add photo + description, generate a draft and explain Python processing.
1:05-1:35	Edit the draft and publish it.
1:35-2:10	Find the product in the marketplace and open details.
2:10-2:35	Submit a buyer inquiry.
2:35-3:00	Show FastAPI /docs and summarize security plus future real AI integration.

Prototype completion condition: one product travels successfully from artisan capture to published marketplace listing and buyer inquiry.